

Recon-ng

Web Reconnaissance Framework — In-Depth Documentation

Prepared for OSINT / penetration testing reference use

1. Overview

Recon-ng is an open-source intelligence (OSINT) gathering tool and full-featured web reconnaissance framework written in Python. It is designed to streamline the process of harvesting information from open sources during the reconnaissance phase of security assessments, ethical hacking engagements, bug bounty hunting, and penetration testing.

It was originally developed by Tim Tomes ("lanmaster53") and is maintained as free, community-supported software.

The tool is frequently described as “the Metasploit of reconnaissance” because it provides an interactive command-line interface that closely resembles the Metasploit Framework workflow: load a module, configure options, run it, review results. This makes it immediately familiar to anyone with penetration testing experience.

Note: Recon-ng focuses exclusively on web-based reconnaissance. It is not designed for exploitation or social engineering — it is a structured data-gathering tool, not an attack framework.

2. Core Architecture

2.1 Workspaces

Every engagement in Recon-ng lives inside a workspace. Workspaces isolate data between targets — each one maintains its own database, module settings, and API key configuration. This means recon data for one client or target never bleeds into another.

2.2 The Database Layer

Recon-ng is not a simple script runner — each workspace has a persistent backing database. Structured tables (domains, hosts, contacts, credentials, and more) store everything the framework collects. Modules read from these tables as input and write newly discovered data back into them as output, allowing findings to chain naturally from one module to the next.

A typical chain: seed the domains table with a target domain → run a subdomain discovery module, which reads that domain and writes new rows into the hosts table → run a further module against those hosts to pull contacts, exposed services, or breach data.

2.3 Modules

The module system is the heart of the framework. Recon-ng ships with roughly 90 modules in total, organized into a few categories:

Category	Approx. Count	Purpose
Recon modules	~76	Domain enumeration, subdomain discovery, contact harvesting, geolocation, credential/breach lookups, social profiling
Reporting modules	8	Export findings to CSV, JSON, HTML, and other report formats
Import modules	2	Pull existing data (e.g. from Nmap, XML, CSV) into the workspace database
Discovery modules	2	More active-touch discovery tasks, as opposed to passive OSINT collection
Exploitation modules	2	Very limited — Recon-ng is not primarily an exploitation framework

Each module is an independent Python class inheriting from a shared base class, which enforces a consistent structure across the entire codebase. Internally:

- `recon/core/framework.py` holds the primary logic for the whole system — it acts as the “brain” that directs how commands are processed and how data flows.
- `recon/core/module.py` handles execution management — overseeing exactly how each individual module runs.

This shared base-class design is what makes the framework easy to extend: a new module only needs to implement its own data-gathering logic, while option-handling, database access, and reporting hooks are inherited for free.

2.4 API Key Management

Many of the most valuable modules depend on third-party services — Shodan, Censys, GitHub, VirusTotal, FullContact, and others — which require API keys. Recon-ng provides a centralized key-management interface so these credentials can be configured once per workspace and reused across every module that needs them. Without keys configured, reconnaissance is limited to free public web scraping and API-free modules.

2.5 Reporting

Findings can be exported in CSV, JSON, or HTML format. A companion interface, `recon-web`, provides built-in analytics and visualization of the data collected in a workspace, which is useful for turning raw findings into a client-ready report.

3. Installation (General Guidance)

Installation steps vary by platform and by the framework's current release, so always cross-check against the official repository before an engagement. As a general pattern:

3.1 Linux / Ubuntu

```
git clone https://github.com/lanmaster53/recon-ng.git cd recon-ng pip install -r REQUIREMENTS
```

3.2 Kali Linux

Recon-ng typically comes pre-installed on Kali Linux, making it immediately available from the terminal.

3.3 Windows

Install via Python and pip directly, or run the Linux installation steps inside Windows Subsystem for Linux (WSL).

4. Typical Workflow

A standard Recon-ng engagement follows a repeatable sequence:

1. Create a workspace — workspaces create <target-name>
2. Seed a domain — insert the target's primary domain into the domains table. The domain acts as the anchor point that later modules reference.
3. Load and run recon modules — for example, a subdomain enumeration module reads the seeded domain and populates the hosts table with what it finds.
4. Chain further modules — use the newly discovered hosts as input for modules that pull contacts, check breach/credential exposure, geolocate IPs, or query Shodan/Censys for exposed services.
5. Report — export results (CSV / JSON / HTML) or review them visually via recon-web.

A commonly used first step that requires no API key is the hackertarget module, which finds hosts associated with a domain using a free public service — a good way to get initial results before configuring any paid API integrations.

4.1 Passive vs. Active Posture

Recon-ng draws a clear line between passive and active-touch modules:

- Recon modules — passive, OSINT-only collection from public sources. These do not touch the target's infrastructure directly and are effectively undetectable by the target's security team.
- Discovery modules — more active techniques that may directly interact with target infrastructure, and are more likely to be logged or noticed.

Note: *If remaining fully passive and undetected is a requirement of the engagement, restrict module usage to the recon category and avoid discovery modules.*

4.2 Automation with Resource Files

Entire recon workflows can be automated using resource files — recorded sequences of Recon-ng commands that can be replayed on demand. This turns a one-off manual recon session into a repeatable pipeline that can be re-run consistently across engagements or targets.

5. Recon-ng vs. SpiderFoot

The two most commonly compared open-source recon frameworks are Recon-ng and SpiderFoot. Both gather OSINT, but they take different approaches:

Aspect	Recon-ng	SpiderFoot
Interface	CLI, Metasploit-style	Web GUI + CLI
Control	Fine-grained, per-module, scriptable	Broad automated correlation
Best for	Precise, targeted collection	Wide automated sweeps
Learning curve	Steeper (practitioner-focused)	Gentler, more guided

A common practitioner pattern is to use both together: Recon-ng for precise, controlled collection against specific targets, and SpiderFoot for broad automated sweeps to surface leads quickly.

6. Practical Considerations & Limitations

- API dependency: the most powerful modules require third-party API keys, and many free tiers have rate limits.
- Module maintenance: Recon-ng's development pace has slowed over time relative to its early years, and some modules can become outdated as the third-party APIs they depend on change their formats or are deprecated. Periodically verify that key modules still function before relying on them for an engagement.
- No GUI: Recon-ng is a CLI-only tool with no built-in onboarding — it is aimed at practitioners already comfortable with a terminal-driven workflow.
- Not an exploitation tool: despite the Metasploit comparison, Recon-ng's exploitation and social-engineering capabilities are minimal to nonexistent by design — use dedicated tools for that phase of an engagement.

7. Part 1 Summary

Recon-ng's core value is turning open-source reconnaissance from a chaotic, manual process — chasing down search engines and copy-pasting results — into a structured, repeatable, and scriptable workflow. Its workspace-and-database model keeps engagements isolated and organized, its modular architecture makes it straightforward to extend, and its resource-file automation makes recon workflows reusable across future targets. For anyone comfortable with a Metasploit-style CLI, it remains one of the most efficient ways to centralize OSINT collection and hand data cleanly from one collection step to the next.

Part 2

The Reconnaissance Phase: Tools, Implementation & Output Validation

8. The Reconnaissance Phase, Structurally

Reconnaissance is phase one of any penetration test or OSINT engagement. It splits into two distinct modes, and the order matters: passive reconnaissance should always precede active techniques, since it is stealthier, legal, and often reveals more than brute-force scanning outright.

- Passive reconnaissance — gathering information about a target without directly interacting with its systems, using only publicly available sources. This makes the activity undetectable to the target's monitoring systems.
- Active reconnaissance — direct interaction with the target's systems to gather more detailed information. These activities can potentially be detected by the target's IDS/IPS or firewall, so they must be run carefully and within agreed test rules.

Note: The intelligence gathered in this phase drives every subsequent phase of the engagement — scanning, enumeration, and exploitation all depend on how thorough and accurate the reconnaissance was.

9. Tools Used, by Mode

The table below consolidates the tools most commonly used across both passive and active reconnaissance, and the specific role each plays:

Tool	Mode	Role
WHOIS / DNS lookup	Passive	Domain registrant data; A, MX, TXT, SRV records reveal infrastructure and mail servers
Google Dorking	Passive	Advanced search operators surface exposed files, login pages, and accidentally published data
theHarvester	Passive	Email address and subdomain harvesting from search engines and public sources
Maltego	Passive	Graph-based OSINT pivoting and relationship mapping using “transforms” against open sources
Shodan / Censys	Passive	Search engines for internet-connected devices; expose already-indexed open services and banners
Amass / Sublist3r / Subfinder	Passive	Broad subdomain enumeration by aggregating many public data sources
httpx	Passive/Active boundary	Confirms which enumerated subdomains actually resolve and respond live
Recon-ng	Passive (mostly)	Orchestration framework tying together many of the above sources into one workspace/database
Nmap	Active	Host discovery, port scanning, service/version detection, OS fingerprinting
Netcat / nmap -sV	Active	Banner grabbing — reads the literal service banner from an open port
VirusTotal / GreyNoise	Supporting / verification	Reputation checks on discovered IPs or files to flag malicious indicators or filter noise

9.1 Passive Recon in Detail

Target: the organization's public footprint (website, WHOIS records, DNS, employees, leaked credentials).

- Website analysis — searching the target's own site for contact information, employee names, technology fingerprints visible in HTML or HTTP headers, and links to other systems.
- Google Dorking — using advanced search operators to find hidden files, login pages, or sensitive information accidentally published on the web.
- WHOIS + DNS enumeration — querying DNS records (A, MX, TXT, SRV, etc.) to identify domains, subdomains, mail servers, and other infrastructure, alongside WHOIS registry lookups for registrant and ownership data.
- theHarvester — automated email address and subdomain enumeration drawn from search engines and other public sources.
- Maltego — a graph-based OSINT tool built around a library of “transforms” that pull data from open sources and visualize it as a link-analysis graph, used for relationship mapping and OSINT pivoting (e.g. domain → employees → social profiles → other owned infrastructure).
- Shodan / Censys — search engines for internet-connected devices that provide real-time intelligence on exposed services, and expose APIs that other tools such as Nmap, Metasploit, and Maltego build on.
- Amass / Sublist3r / Subfinder — comprehensive DNS reconnaissance and subdomain enumeration by aggregating many public data sources, typically piped into a liveness checker such as httpx afterward.
- Recon-ng — the orchestration framework detailed in Part 1 of this document; ties many of the sources above together inside one workspace/database-backed workflow rather than running each tool standalone.

9.2 Active Recon in Detail

Once passive reconnaissance has produced a list of live hosts and subdomains, active reconnaissance confirms and details what is actually running on them.

- Nmap — the universal baseline for network discovery and service identification. Host discovery is rarely performed in isolation; it is typically a preparatory step feeding into port scanning, service/version detection, and OS fingerprinting.
- Banner grabbing (Netcat or nmap -sV) — reads the literal service banner returned by an open port. In one documented case, banner grabbing returned a vsFTPD 2.3.4 banner; recognizing that this version had a known backdoor let the tester move straight to Exploit-DB and obtain a remote shell within minutes — a clear illustration of recon output feeding directly into exploitation.

10. How Implementation Is Chained Together

In practice, tools are not run once in isolation — the realistic workflow is a pipeline where each tool's output becomes the next tool's input:

1. Scope the target — domain, IP range, or organization name.
2. Passive collection — WHOIS and DNS enumeration, plus theHarvester / Amass / Sublist3r, to build an initial subdomain list.

3. Liveness filtering — pipe the subdomain list into httpx (or similar) to confirm which hosts actually respond.
4. Exposure cross-reference — check confirmed live hosts against Shodan/Censys for already-indexed exposed services and known CVEs.
5. Active scanning — run Nmap against the confirmed live hosts for detailed port, service, and version information.
6. Banner grab and verify — grab banners on interesting services and cross-check versions against Exploit-DB, VirusTotal, or GreyNoise for known issues.
7. Consolidate — merge all findings into a single structured document before moving into the exploitation phase.

Automation matters at every one of these steps: simple scripts that loop Nmap across subnets or run theHarvester across a list of domains save significant time, and both Shodan and Censys expose APIs that can be queried directly and filtered programmatically.

11. Validating and Debugging the Output

Raw reconnaissance output is noisy by default — dead subdomains, stale DNS records, false-positive “open” ports behind a firewall decoy, and expired WHOIS data are all common. Treating the output as provisional until verified is an essential step, not an optional one.

- Dedupe and liveness-check — always filter out duplicates and non-responsive hosts before trusting a subdomain list; a name resolving in DNS does not mean the host is actually reachable.
- Cross-validate across sources — if Shodan reports an exposed service that a direct Nmap scan does not confirm, that discrepancy is a signal the indexed data may be stale rather than a signal the service is still exposed.
- Reputation-check anything suspicious — services such as VirusTotal or GreyNoise can be queried to check whether a discovered IP or file is malicious, which also helps distinguish genuine findings from routine internet background-scan noise.
- Re-run with different scan techniques when results look wrong — if a default Nmap scan shows everything as filtered, that is often a firewall rather than a genuinely closed host; adjusting scan type or timing frequently reveals the real state.
- Log everything with timestamps — maintaining a detailed log of timestamps, tool outputs, and observations ensures traceability and repeatability, which is what allows a strange result to be debugged accurately days later.
- Structure findings before handoff — categorize collected data systematically, and use spreadsheets, mind maps, or tools such as Maltego to visualize relationships and dependencies, so the final report clearly separates confirmed findings from items still pending re-verification.

12. Overall Summary

Reconnaissance succeeds or fails on discipline, not tooling alone: working passive-before-active, chaining each tool's output into the next tool's input, and treating every raw result as provisional until cross-verified. Recon-ng's role in this larger picture is as an orchestration layer — a way to run many of these individual passive-recon sources (DNS, Shodan, contact harvesting, and more) from a single, database-backed, scriptable workflow rather than juggling a dozen standalone tools by hand. Used well, it shortens the path

from a bare domain name to a validated, well-documented attack-surface picture ready for the next phase of an engagement.